

1. [2p] Dado el siguiente código Java,

- Dibujar un diagrama de clases UML que muestre todas las clases e interfaces involucradas y sus relaciones.
- Si se desea agregar un nuevo tipo de ítem Servicio (que calcule su precio según horas y tarifa), explique brevemente por qué este diseño respeta el Principio de Abierto/Cerrado (Open/Closed Principle).

Vendible.java

```
1 public interface Vendible {
2     double getPrecio();
3 }
```

Producto.java

```
1 public class Producto implements Vendible {
2     private double precio;
3     public Producto(double p) { precio = p; }
4     public double getPrecio() { return precio; }
5 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Combo c1 = new Combo(0.1);
4         c1.add(new Producto(100));
5         Combo c2 = new Combo(0.2);
6         c2.add(new Producto(50));
7         c1.add(c2);
8         System.out.println(c1.getPrecio());
9     }
10 }
```

Combo.java

```
1 public class Combo implements Vendible {
2     private List<Vendible> items = new ArrayList<>();
3     private double descuento;
4
5     public Combo(double d) { descuento = d; }
6     public void add(Vendible v) { items.add(v); }
7
8     public double getPrecio() {
9         double total = 0;
10        for (Vendible v : items) {
11            total += v.getPrecio();
12        }
13        return total * (1 - descuento);
14    }
15 }
```

#### Criterios de corrección:

- **Diagrama de clases:**

- Producto implementa Vendible (realización)
- Combo implementa Vendible (realización)
- Combo también tiene una relación de agregación con Vendible (1 a muchos)
- Main dependiente de Combo y Producto (uso)

- **OCP:** Se debe mencionar que para agregar Servicio no es necesario modificar las clases existentes (Combo, Producto, Vendible), sino que se extiende el sistema creando una nueva clase que implementa la interface.

## 2. [2p]

Dada la siguiente base de datos Prolog:

- a) ¿Qué resultados devuelve la consulta `?- recomendada(toy_story, X)?`  
Indicar todas las soluciones en el orden en que Prolog las encontraría, y justificar.
- b) Definir un predicado `colegas(D1, D2)` que sea cierto si D1 y D2 son directores distintos que dirigieron alguna película del mismo género (por ejemplo, `colegas(nolan, wachowski)` debería ser cierto).

`peliculas.pl`

```
1 pelicula(toy_story, animacion, lassetter).
2 pelicula(cars, animacion, lassetter).
3 pelicula(pulp_fiction, crimen, tarantino).
4 pelicula(kill_bill, crimen, tarantino).
5 pelicula(interstellar, ciencia_ficcion, nolan).
6 pelicula(matrix, ciencia_ficcion, wachowski).
7
8 recomendada(P1, P2) :-
9     pelicula(P1, G, _),
10    pelicula(P2, G, _),
11    P1 \= P2.
12
13 recomendada(P1, P2) :-
14     pelicula(P1, _, D),
15     pelicula(P2, _, D),
16     P1 \= P2.
```

### Criterios de corrección:

- `X = cars` (por la primera regla, mismo género) y luego `X = cars` nuevamente (por la segunda regla, mismo director). Es importante que noten que la solución aparece dos veces.
- `colegas(D1, D2) :- pelicula(_, G, D1), pelicula(_, G, D2), D1 \= D2.`

3. [2p] Dado el siguiente código Clojure:

`sensores.clj`

```
1 (def mediciones
2   [{:sensor "A" :temp 20}
3    {:sensor "B" :temp 25}
4    {:sensor "A" :temp 22}
5    {:sensor "C" :temp 18}])
6
7 (defn misterio1 [limite datos]
8   (filter #( > (:temp %) limite) datos))
9
10 (defn misterio2 [datos]
11   (map :sensor datos))
12
13 (defn misterio3 [datos]
14   (let [ts (map :temp datos)]
15     (/ (reduce + ts) (count ts))))
```

- a) Explicar, utilizando conceptos y terminología de programación funcional, qué hace `misterio1`. ¿Cuál sería un nombre más descriptivo para esta función?
- b) Repetir el punto anterior para `misterio2`.
- c) Repetir el punto anterior para `misterio3`.
- d) ¿Son estas funciones “puras”? Justificar brevemente analizando si dependen de algún estado global o si producen efectos colaterales.

**Criterios de corrección:**

- **misterio1:** Es una función de orden superior que aplica un filtro sobre una colección. Un nombre posible sería `filtrar-por-temperatura` o `filtrar-altas`.
- **misterio2:** Es una función de orden superior que aplica una transformación (mapeo) sobre la colección para extraer los nombres de los sensores. Un nombre posible sería `nombres-sensores`.
- **misterio3:** Calcula el promedio de las temperaturas. Utiliza `map` para extraer los valores y `reduce` para sumarlos. Un nombre posible sería `promedio-temperaturas`.
- **Pureza:** Sí, son puras. No tienen efectos colaterales (como I/O) y su resultado depende exclusivamente de los argumentos de entrada. No modifican las estructuras de datos originales (inmutabilidad).

4. [2p] Dada la siguiente expresión de cálculo lambda:

$$((\lambda x. \lambda y. x \ y) (\lambda z. z)) \ a$$

- Identificar qué variables son libres y cuáles son ligadas en la expresión.
- Reducir la expresión a su forma normal, mostrando todos los pasos intermedios de la reducción.

**Criterios de corrección:**

- **Variables:** x, y y z son ligadas (aparecen como parámetros de un  $\lambda$ ). a es una variable libre (no está bajo el alcance de ningún  $\lambda$ ).
- **Reducción:**
  - $((\lambda x. \lambda y. x \ y) (\lambda z. z)) \ a$
  - $(\lambda y. (\lambda z. z) \ y) \ a$  ( $\beta$ -reducción de x por  $(\lambda z. z)$ )
  - $(\lambda z. z) \ a$  ( $\beta$ -reducción de y por a dentro del cuerpo anterior, o bien reducción interna de  $(\lambda z. z)$  y primero).
  - a ( $\beta$ -reducción final).

5. [1p] Dado el siguiente fragmento de código de una aplicación JavaFX:

```
1 public void start(Stage stage) {  
2     Slider slider = new Slider(0, 100, 50);  
3     Label label = new Label();  
4  
5     label.textProperty().bind(  
6         slider.valueProperty().asString("Valor: %.0f")  
7     );  
8  
9     VBox root = new VBox(slider, label);  
10    stage.setScene(new Scene(root, 200, 100));  
11    stage.show();  
12 }
```

- a) Dibujar el **Scene Graph** resultante de la ejecución del método start.
- b) Explicar en términos de **programación orientada a eventos** y **programación gráfica**: ¿Qué característica de JavaFX se aprovecha en las líneas 5-7? ¿Qué ocurre cuando el usuario desplaza el control slider? ¿Qué efecto tiene en el label?

**Criterios de corrección:**

- **Scene Graph:** Debe mostrar la jerarquía: Stage -> Scene -> VBox (root) -> [Slider, Label].
- **Comportamiento:** El texto del label se actualiza automáticamente para mostrar el valor actual del slider (formateado). Se logra mediante el mecanismo de **Bindings** de JavaFX, donde la propiedad de destino (textProperty) se suscribe a los cambios de la propiedad de origen (valueProperty). Internamente JavaFX suscribe un **listener** al slider, y cada vez que su valor cambia, se recalcula el texto del label y se actualiza en la interfaz.

6. [1p] Dado el siguiente código Clojure:

```
1 (def contador (atom 0))
2
3 (defn incrementar []
4   (swap! contador inc))
5
6 (doall (pcalls (repeat 100 incrementar)))
7
8 (println @contador)
```

- a) Explicar en términos de **programación funcional** y **programación concurrente** qué hace el código y cuál es el valor final impreso en la última línea.
- b) ¿Por qué se utiliza un `atom` y la función `swap!` para gestionar el valor de contador en lugar de una variable común?
- c) ¿Es determinístico el valor impreso en la última línea? Justifique brevemente analizando si existe riesgo de condiciones de carrera.

**Criterios de corrección:**

- **Comportamiento:** Se define un contador compartido inicializado en 0.
  - La función `incrementar` incrementa el valor del contador de manera atómica.
  - `(repeat 100 incrementar)` genera una secuencia de 100 llamadas a `incrementar`.
  - `pcalls` ejecuta estas llamadas en forma concurrente.
  - `doall` fuerza la evaluación de todas las llamadas concurrentes (ya que `pcalls` devuelve una secuencia perezosa).
  - Finalmente, se imprime el valor del contador, que será 100.
- **Uso de atom/swap:** Los atoms en Clojure proporcionan una forma de gestionar estado compartido, mutable y sincrónico de manera segura. `swap!` garantiza que la actualización sea atómica, evitando que varios hilos interfieran entre sí al leer y escribir.
- **Determinismo:** Sí, el valor final será siempre 100. Al usar `swap!`, cada incremento se realiza de forma atómica. Si dos hilos intentan actualizar al mismo tiempo, uno de ellos reintentará la operación con el nuevo valor, garantizando que no se pierda ningún incremento (no hay condiciones de carrera que afecten el resultado final).